

DataShare

Application de partage de fichiers sécurisé via liens temporaires

Noémie Henry — Soutenance Projet 2026

Contexte & Objectif

Problème

Partager des fichiers volumineux sans serveur centralisé ni compte requis

Solution

Une app web permettant l'upload et le partage via liens temporaires uniques

Cible

Utilisateurs souhaitant partager des fichiers facilement et en toute sécurité

Stack technique

Backend

NestJS + TypeScript

Architecture modulaire, JWT auth, Multer upload

Frontend

React + Vite + TypeScript

SPA accessible (WCAG) et responsive, React Context

Base de données

PostgreSQL

TypeORM, synchronisation auto, Docker

Stockage

Système de fichiers local

Fichiers stockés en local dans /uploads

Tests

Jest + 98,7% coverage

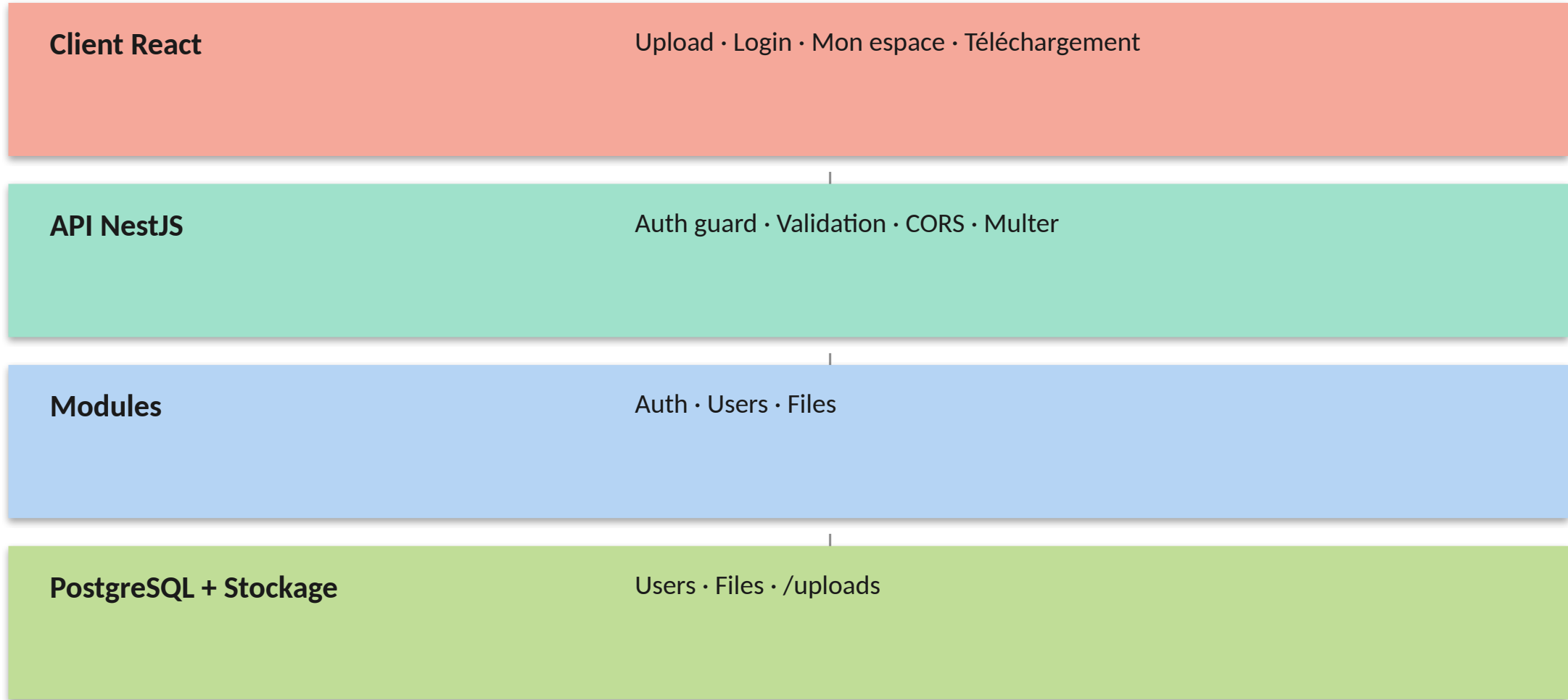
58 tests : 47 unit · 7 e2e API · 4 Cypress

Déploiement

Docker Compose

Setup en 1 commande avec setup.sh

Architecture de la solution



Fonctionnalités MVP

US01 Upload fichier

Fichier jusqu'à 1Go, mot de passe optionnel, expiration 1-7j

US02 Téléchargement

Lien unique non prédictible, métadonnées visibles avant dl

US03 Création compte

Email unique, password hashé bcrypt, token JWT

US04 Connexion

JWT en cookie HttpOnly, valide 7 jours

US05 Historique

Filtres Tout/Actifs/Expirés, liste avec dates

US06 Suppression

Suppression physique + BDD, confirmation requise

Qualité & Tests

58

tests au total

98,7%

couverture de code

100%

auth.service

98,7%

files.service

TESTING.md

Plan de tests, couverture 98,7%, critères d'acceptation

SECURITY.md

bcrypt, JWT, CORS, tokens UUID, validation type fichier, npm audit

PERF.md

Tests k6, métriques temps de réponse, budget bundle

MAINTENANCE.md

Procédures mises à jour, backup BDD, risques

Difficultés & Solutions

Conflit type File / FileEntity

Pb : TypeScript confondait l'entité avec le type global File du navigateur

Sol : Renommage en FileEntity + imports explicites

JWT secret inconsistant

Pb : Token signé avec un secret différent de celui de la vérification

Sol : Secret déplacé dans le .env (ConfigService), lu à la signature et à la vérification

PostgreSQL local vs Docker

Pb : Mac avait un PostgreSQL local qui interceptait le port 5432

Sol : Port Docker changé en 5433 dans docker-compose et .env

Coverage à 13%

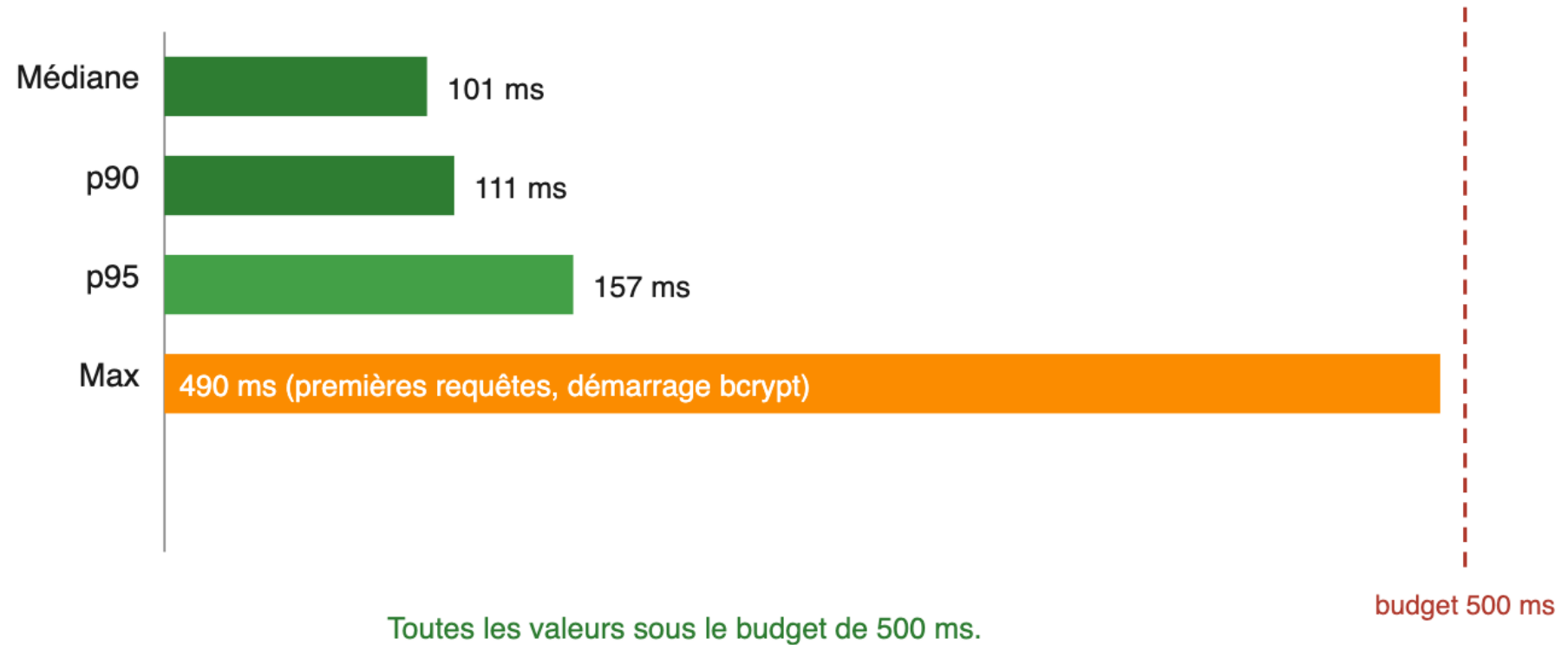
Pb : Fichiers de config (modules, main.ts) faisaient baisser la moyenne

Sol : Tests ajoutés sur strategy, guard et intercepteur → couverture 98,7 %

Performance

Test de charge k6 — POST /auth/login

10 utilisateurs en parallèle, 30 s — 100 % de succès



Route la plus coûteuse (bcrypt). Test k6 ; bundle front gzippé < 100 Ko ; logs structurés JSON.

Sécurité

- Mots de passe hachés avec bcrypt (10 rounds) — jamais stockés en clair.
- JWT dans un cookie HttpOnly (anti-XSS) ; secret en variable d'environnement, jamais en dur.
- Liens de téléchargement = UUID v4 non devinables, avec expiration (1 à 7 jours).
- Refus des exécutables par leur contenu réel (magic number), pas l'extension.
- Cloisonnement : un utilisateur n'agit que sur ses propres fichiers.
- Limite de taille 1 Go (anti-DoS) ; CORS restreint ; scan npm audit documenté.

Retour du mentor & améliorations

- « Trop superficielle » → présentation enrichie, code expliqué en profondeur.
- Secret JWT en dur → sorti dans le .env (ConfigService).
- Conteneurisation incomplète → front + back + base dans Docker (+ /health).
- 2 fichiers à 0 % de couverture → testés (strategy, guard) → couverture 98,7 %.
- Tests de performance et de sécurité → désormais mis en avant (slides dédiées).
- SOLID / revue de code → expliqués (modules, injection de dépendances).